

CHRONOS

Turning a Personal Frustration into a Product

Personal Product Project

Android Application

Automating repetitive, time-sensitive communication through a simple scheduling experience.

Nikhil

Kotlin • Jetpack Compose • Android

[GitHub Repository](#)

1. The Story Behind Chronos

The idea for Chronos came from a personal experience.

One day, my phone broke, and I missed a call from my father. After that incident, he started asking me to send him good morning and good night messages every day. While the intention behind those messages was simple, remembering to send them consistently and at the right time became a tedious daily task.

I often missed the timing, which led to repeated reminders from my father that I wasn't messaging him regularly.

Instead of continuing to manage this manually, I started thinking about a way to automate the process.

Initially, my goal was very specific:

I wanted a simple way to send good morning and good night messages to my father without having to remember to do it every day.

That small, personal inconvenience became the starting point for Chronos.

2. Identifying the Problem

The problem wasn't writing a message. It was remembering to send it at the right time, every single day.

Traditional alarms and reminders could notify me when it was time to send a message, but I would still have to stop what I was doing, open WhatsApp, find the conversation, and send it manually.

I wanted a solution that could handle the task itself, rather than simply reminding me to perform it.

2.1 The Core Problem

How might I automate repetitive, time-sensitive messaging so that people can stay consistent with their communication without having to remember and manually send every message?

This became the central problem I wanted Chronos to solve.

3. Exploring Existing Solutions

Before building anything, I explored the apps already available on the Google Play Store.

I found several apps that offered message scheduling and automation. However, many of

the apps I explored required Accessibility permissions to interact with the phone's user interface.

Granting such broad access to a third-party application raised concerns for me about privacy, control, and trust. I was uncomfortable giving an unfamiliar application that level of access to my device.

This became an important consideration in my approach.

Rather than relying on an unfamiliar third-party application, I decided to build my own solution. Developing it myself would give me greater visibility into how it worked, how it handled permissions, and what it was doing on my device.

The goal wasn't simply to recreate an existing app. It was to build a solution that I could understand and feel comfortable using myself.

4. From a Personal Use Case to a Broader Opportunity

Initially, Chronos was intended only for sending daily good morning and good night messages to my father.

However, as I thought more about the idea, I realized that the underlying problem wasn't limited to personal greetings. Many people have messages they need to send at a particular time but may forget to send them when the time comes.

I identified several other potential use cases:

4.1 Group Communication

Scheduling a particular message to be sent to a WhatsApp group at a specific time, without having to remember to send it manually.

4.2 Group Reminders

Sending reminders to a group when people need to be informed about an upcoming activity, meeting, or deadline.

4.3 Routine Communication

Automating recurring messages that otherwise require the same manual effort every day or week.

4.4 Time-Sensitive Messages

Preparing a message in advance so that it can be triggered at the intended time, even when the sender is asleep or away from their phone.

This changed how I viewed the project. What began as a solution for one personal inconvenience had the potential to become a more general-purpose scheduling tool.

5. Defining the Product

I wanted Chronos to go beyond the functionality of a conventional alarm or reminder.

An alarm tells a person when to do something. Chronos was designed to schedule the communication task itself.

5.1 Product Vision

To make repetitive and time-sensitive messaging easier by allowing users to prepare messages in advance and automate the scheduling process.

5.2 Core User Workflow

The core experience was built around a simple workflow:

1. **Create a message:** Write the message that needs to be sent.
2. **Select a recipient:** Choose the intended WhatsApp contact or group.
3. **Choose a schedule:** Select a date, time, or recurring schedule.
4. **Manage the schedule:** View, edit, pause, resume, or delete scheduled messages.
5. **Automate the workflow:** Let Chronos trigger the messaging process at the scheduled time and provide feedback about the action.

This approach aimed to reduce the number of manual steps required from the user while keeping scheduled messages manageable.

6. Building the Solution

After defining the problem and the broader use cases, I decided to develop Chronos as an Android application.

I used Kotlin and Jetpack Compose for the application, with local storage and Android scheduling capabilities to support scheduled tasks and message management.

6.1 Core Features

Feature	Description
Message Scheduling	Preparing messages for a specific date and time.
Recurring Messages	Setting up repeated schedules for routine communication.
Recipient and Group Selection	Choosing where the message should be directed.
Schedule Management	Viewing, editing, pausing, resuming, and deleting scheduled messages.
Automated Workflow	Triggering the messaging process at the scheduled time, subject to Android permissions, device conditions, and WhatsApp’s interface.
Status and Notifications	Informing users about scheduled actions and cases where an action may have failed.

Table 1: Core capabilities of Chronos

6.2 Development Approach

I developed the application independently, taking responsibility for the product idea, feature decisions, interface, and implementation.

My focus was on creating a functional application that addressed the original problem while remaining flexible enough to support other scheduling scenarios.

7. Challenges and Product Trade-offs

One of the main challenges was balancing automation, user trust, and reliability.

7.1 Permissions and User Trust

The existing solutions I explored had raised concerns about granting Accessibility permissions to third-party applications.

Building my own app gave me greater visibility into its implementation, but it did not remove the need to handle Android permissions responsibly.

I had to consider how the application would interact with the device while minimizing unnecessary access and maintaining user control.

7.2 Android Background Restrictions

Android imposes background execution restrictions that can affect when and how applications perform scheduled tasks.

Device conditions, battery optimization, permissions, and operating system behavior can all influence the reliability of scheduled actions.

This meant that triggering an action at an exact time could not always be guaranteed under every device condition.

7.3 WhatsApp Automation Limitations

An application can trigger an action at a scheduled time without necessarily being able to guarantee that WhatsApp will successfully send or deliver the message.

Changes to WhatsApp's interface, device state, and permission settings can affect the automation workflow.

This highlighted an important product consideration:

Users should not be led to believe that a message was delivered if the application can only confirm that it attempted to perform the action.

7.4 Communicating Failures

A useful scheduling application needs to do more than initiate an action. It must also communicate what happened.

I therefore had to consider not only how to schedule messages, but also how to manage failures and communicate the limitations of automation.

8. Outcome

Chronos evolved from a solution for a single personal use case into a working Android application intended to support a wider range of scheduled messaging needs.

The project allowed me to take an idea rooted in my own experience and turn it into a functioning product.

It also gave me practical exposure to:

- Product ideation and problem identification.
- Understanding user needs and pain points.
- Identifying adjacent use cases and product opportunities.
- Defining features and designing user workflows.
- Android application development.
- Navigating technical constraints and making product trade-offs.

- Considering privacy, trust, and reliability as part of the user experience.

Most importantly, Chronos helped me understand how a small, everyday frustration can become the foundation for a broader product idea.

9. Key Learnings

Building Chronos changed how I think about everyday problems and product opportunities.

9.1 Start with a Real Problem

A small inconvenience in my daily life became the starting point for a product idea. It reinforced the value of observing recurring problems in everyday experiences instead of searching for ideas without a clear user need.

9.2 Look Beyond the Initial User

Solving a problem for one person helped me recognize similar needs in other communication scenarios. A solution designed around a specific use case can reveal opportunities to serve a wider audience.

9.3 Consider Trust as Part of the Experience

A product that handles personal communication must make users comfortable with its permissions and behavior. Privacy, transparency, and user control are not just technical considerations. They are part of the product experience.

9.4 Automate the Task, Not Just the Reminder

There is a meaningful difference between telling users what they need to do and helping them complete it. Chronos was an attempt to reduce the manual effort involved in routine communication rather than simply adding another reminder.

9.5 Account for Real-World Limitations

A useful product must consider failure cases and communicate honestly about what it can guarantee. Technical feasibility alone is not enough. Reliability and clear feedback are equally important to a good user experience.

10. What I Would Do Next

If I were to continue developing Chronos, my next priority would be to validate the broader use cases with actual users.

10.1 User Validation

- Test whether people find message scheduling genuinely useful.
- Identify which types of messages users schedule most often.
- Understand how much users trust automated messaging.
- Gather feedback on the scheduling and management workflow.

10.2 Reliability Improvements

- Evaluate scheduled actions across different Android devices and operating system versions.
- Improve failure detection and feedback.
- Investigate the effects of background restrictions and battery optimization.
- Make the status of scheduled actions more transparent.

10.3 Product Development

- Prioritize features based on actual user feedback.
- Improve the overall scheduling experience.
- Explore additional recurring messaging scenarios.
- Identify ways to make automation more reliable while respecting user control and platform limitations.

The next stage would be to move beyond solving a problem I personally experienced and establish how well Chronos solves that problem for a wider group of users.

11. Tech Stack

12. Project Repository

The source code and project implementation are available on GitHub.

Chronos-Android Repository

<https://github.com/Nephophile-Nik/Chronos-Android>

Technology	Purpose
Kotlin	Primary programming language
Jetpack Compose	Modern Android UI development
Android Scheduling APIs	Scheduling and triggering tasks
Local Storage	Managing scheduled messages and application data
WhatsApp	Intended messaging platform

Table 2: Technologies used in Chronos

Chronos started as a way to remember less and communicate more consistently. It became an opportunity to explore how personal problems can inspire meaningful products.

CHRONOS